

UNIVERSIDAD RAFAEL LANDIVAR
FACULTAD DE INGENIERIA
LENGUAJES FORMALES Y AUTOMATAS
ING. ROJAS ARGUETA CESAR SALVADOR

MANUAL TECNICO

Proyecto 2

INTEGRANTES:

RODRIGO RUIZ 1037623

JAROD BOLAÑOS 1251621

CIUDAD DE GUATEMALA

DESCRIPCION DEL SISTEMA

El sistema es una herramienta de software desarrollada en Python 3 (modo consola) orientada al modelado, análisis formal, simulación y conversión de autómatas.

Evolucionando la arquitectura de la Fase 1 sin alterar sus capacidades operativas, el sistema permite instanciar tanto **Autómatas Finitos Deterministas (AFD)** como **Autómatas Finitos No Deterministas (AFND)** mediante ingreso manual interactivo o por lectura y parsing de archivos planos de texto (.txt) utilizando expresiones regulares.

A partir de un AFND cargado y validado, el sistema aplica el algoritmo de **construcción de subconjuntos** para transformarlo en un AFD equivalente, registrando la tabla de equivalencias entre macroestados y habilitando la reutilización completa de las herramientas de validación, análisis estructural, traza de ejecución e historial sobre el AFD generado. Todo el proyecto está desarrollado mediante estructuras nativas de Python (set, dict, list), sin depender de librerías externas de autómatas.

REQUISITOS DE LA EJECUCION DEL SOFTWARE

- **Lenguaje:** Python 3.10 o superior.
- **Módulos Estándar Utilizados:** sys, os, re, dataclasses (no requieren instalación adicional vía pip).
- **Sistemas Operativos Compatibles:** Microsoft Windows 10/11, GNU/Linux, macOS.
- **Codificación de Consola:** Compatibilidad nativa e interactiva UTF-8 mediante reconfiguración explícita de sys.stdin, sys.stdout y sys.stderr (para renderizar y leer símbolos formales como δ , λ , ϵ y $\emptyset$).

ARQUITECTURA DEL SOFTWARE

El proyecto está diseñado bajo el principio de separación de responsabilidades dividiendo el sistema en módulos independientes:

Módulo	Descripción / Funcionalidad
main.py	Punto de entrada del sistema. Contiene el bucle principal, el menú de 15 opciones y la orquestación/reglas de acceso entre módulos sin asumir responsabilidades de carga o conversión.
automata.py	Define las clases base AFD y AFND. Encapsula la quintupla formal $M = (Q, \Sigma, \delta, q_0, F)$ y provee métodos de consulta.
cargador.py	Implementa el motor de parsing con expresiones regulares para la creación interactiva y lectura de archivos .txt de un AFD.
cargador_afnd.py	Procesa el parsing interactivamente y desde .txt para AFND, gestionando transiciones con múltiples destinos y el símbolo de conjunto vacío ($\emptyset$).
validador.py	Verifica la integridad estructural del AFD, completitud, transiciones faltantes, estado de trampa y análisis de alcance/lenguaje vacío.
validador_afnd.py	Revisa la coherencia interna de las definiciones de un AFND y valida la consistencia de sus conjuntos de llegada.
conversor.py	Implementa el algoritmo de construcción de subconjuntos para transformar un AFND en un AFD equivalente, generando la tabla de macroestados.
simulador.py	Ejecuta la simulación de cadenas individuales o en lotes sobre un AFD, genera trazas paso a paso y registra eventos.
colores.py	Módulo auxiliar con secuencias de escape ANSI para formato visual de consola.

MODELADO DE DATOS E IMPLEMENTACION TECNICA

Abstracción de la Quintupla (Clase AFD)

La clase AFD en automata.py encapsula los elementos de la quintupla $M = (Q, \Sigma, \delta, q_0, F)$

class AFD:

```
def __init__(self, nombre, Q, sigma, delta, q0, F):
    self.nombre = nombre      # Identificador legible (str)
    self.Q = set(Q)           # Conjunto de estados Q (set[str])
    self.sigma = set(sigma)   # Alfabeto Sigma (set[str])
    self.delta = dict(delta)  # Función delta: dict[(str, str), str]
    self.q0 = q0              # Estado inicial q0 (str)
    self.F = set(F)           # Conjunto de estados finales F (set[str])
    self.validado = False     # Estado de validación (bool)
```

class AFND:

```
def __init__(self, nombre, Q, sigma, delta, q0, F):
    self.nombre = nombre      # Identificador legible (str)
    self.Q = set(Q)           # Conjunto de estados Q (set[str])
    self.sigma = set(sigma)   # Alfabeto Sigma (set[str])
    self.delta = dict(delta)  # Función delta: dict[(str, str), set[str]]
    self.q0 = q0              # Estado inicial q0 (str)
    self.F = set(F)           # Conjunto de estados finales F (set[str])
    self.validado = False     # Estado de validación (bool)
```

Función de Transición (δ) sin Condicionales

Para dar cumplimiento a lo que nos indica el proyecto, en la función de transición no utilizamos estructuras condicionales anidadas (if/else o switch). En su lugar, utilizamos la estructura de diccionario nativo (dict):

- **Clave de acceso:** Tupla (estado_origen, simbolo).
- **Valor retornado:** Cadena estado_destino.
- **Mecanismo:** El método `self.delta.get((estado, simbolo))` evalúa la transición en tiempo constante $O(1)$ para cualquier autómatas dinámico.

Para evitar cadenas de if/elif:

- **En AFD:** delta almacena tuplas (estado_origen, simbolo) -> estado_destino. La consulta se realiza vía `delta.get((q, a))` en tiempo constante $O(1)$.
- **En AFND:** delta almacena tuplas (estado_origen, simbolo) -> `set[estado_destino]`. Si no hay transiciones explícitas, el resultado retornado es un conjunto vacío $\emptyset$ (`set()`).

SIMBOLOS REGEX UTILIZADOS

Símbolo Regex	Nombre Técnico	Función en el Parsing del Archivo
^	Ancla de inicio	Exige que la coincidencia comience exactamente desde el primer carácter de la línea leída.
\$	Ancla de fin	Obliga a que la evaluación termine al final de la línea, evitando que se acepte texto "basura" adicional.
.	Punto (Comodín)	Representa cualquier carácter individual (excepto un salto de línea).
*	Cuantificador (Cero o más)	Permite que el elemento anterior aparezca muchas veces o ninguna (útil para listas de estados finales vacías).
+	Cuantificador (Uno o más)	Obliga a que el elemento anterior aparezca al menos una vez (ej. el nombre del autómatata no puede estar vacío).
(...)	Grupo de captura	Extrae la subcadena exacta que hace <i>match</i> para almacenarla en variables dentro de Python.
[...]	Clase de caracteres	Define un rango de caracteres permitidos (ej. <code>[A-Za-z0-9_]</code> obliga a que los nombres de estados sean alfanuméricos).
\s	Espacio en blanco	Coincide co-espacios o tabulaciones. Combinado con <code>*</code> (<code>\s*</code>), permite ignorar espacios irregulares alrededor de las comas.
[^...]	Clase negada	Coincide con cualquier carácter que no esté en la lista (ej. <code>[^\s,]</code> lee el símbolo del alfabeto asegurando que no sea un espacio ni una coma)

MOTOR DE PARSING (EXPRESIONES REGULARES)

El módulo cargador.py utiliza el módulo re para validar la sintaxis de cada línea del archivo .txt. Las expresiones regulares implementadas son:

- Nombre del Autómata: `^NOMBRE=(.+)$`
- Conjunto de Estados (Q): `^ESTADOS=(.+)$`
- Alfabeto (Σ): `^ALFABETO=(.+)$`
- Estado Inicial (q_0): `^INICIAL=(.+)$`
- Estados Finales (F): `^FINALES=(.*)$`
- Encabezado de Transiciones: `^TRANSICIONES:$`
- Definición de Transición: `^([A-Za-z0-9_]+)\s*,\s*([^\s,]+)\s*,\s*([A-Za-z0-9_]+)$`

VALIDACIONES TEÓRICAS Y ANÁLISIS ESTRUCTURAL

Validaciones de Integridad (Funcionalidad 2)

1. Pertenencia del Estado Inicial: Se verifica que $q_0 \in Q$.
2. Subconjunto de Aceptación: Se verifica que $F \subseteq Q$ mediante operaciones de conjuntos ($F.\text{issubset}(Q)$).
3. Pertenencia de Transiciones: Se asegura que cada transición $(q, a) \rightarrow p$ cumpla con $q \in Q$, $p \in Q$ y $a \in \Sigma$.
4. Verificación Determinista: Se garantiza que no existan múltiples destinos para una misma clave (q, a) ni se permitan transiciones con símbolos épsilon (ϵ, λ).

Análisis Estructural de Grafos

Mediante un algoritmo de recorrido comenzando en q_0 :

- Estados Alcanzables: Subconjunto de Q accesible mediante uno o más saltos desde q_0 .
- Estados Inaccesibles: Calculados mediante la diferencia de conjuntos $Q - \text{Alcanzables}$.
- Detección de Lenguaje Vacío: Se evalúa si $\text{Alcanzables} \cap F = \emptyset$. Si la intersección es vacía, el autómata no puede aceptar ninguna cadena.

MANEJO DE EXCEPCIONES Y RESILIENCIA

El sistema previene el colapso ante errores mediante las siguientes estrategias:

- Entrada/Salida de Archivos: Captura explícita de `FileNotFoundError` y `OSError` al intentar cargar rutas inválidas.
- Interrupción de Usuario: Captura de `KeyboardInterrupt` (Ctrl+C) y `EOFError` en el punto de entrada `main.py` para realizar cierres limpios sin mostrar trazas de error (*tracebacks*) al usuario.
- Reconfiguración de Codificación: Ejecución de `sys.stdout.reconfigure(encoding="utf-8", errors="replace")` para evitar colapsos por `UnicodeEncodeError` en consolas de comandos de Windows.

ALGORITMO DE CONSTRUCCIÓN DE SUBCONJUNTOS (FASE 2)

El módulo conversor.py implementa la transformación de AFND a AFD de forma genérica:

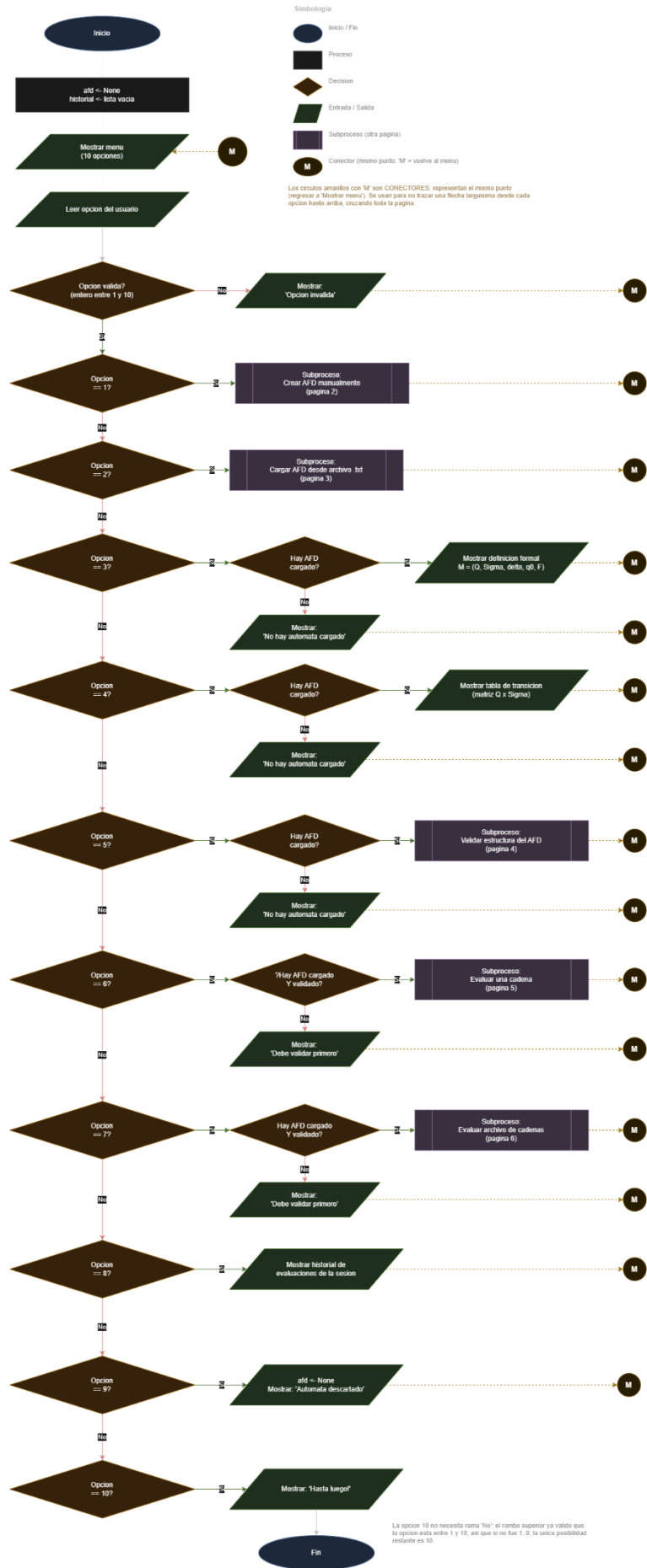
1. **Estado Inicial:** Se crea el macroestado inicial $S_0 = \{q_0\}$.
2. **Cola de Procesamiento:** Se mantiene una lista de macroestados pendientes por evaluar.
3. **Mapeo de Transiciones:** Para cada macroestado S pendiente y cada símbolo $a \in \Sigma$:

$$S' = \bigcup_{q \in S} \delta_{AFND}(q, a)$$

4. **Tratamiento del Conjunto Vacío ($\emptyset$):** Si $S' = \emptyset$, este se registra explícitamente como un macroestado de trampa $C = \emptyset$ dentro del AFD generado.
5. **Determinación de Estados Finales:** Un macroestado S se agrega al conjunto F_{AFD} si $S \cap F_{AFND} \neq \emptyset$.
6. **Mapeo de Nombres / Tabla de Equivalencias:** Cada subconjunto S se nombra con una etiqueta alfabética ($A, B, C, \dots$) y se almacena en una estructura de equivalencia:

Tabla de Equivalencias: Macroestado AFD $\Leftrightarrow$ {Estados del AFND}

Una vez que se a estructurado la forma del programa y que es lo que utiliza cada una de las partes procederemos a explicar como funciona el programa de manera sencilla, al tratar de ejecutar el programa se puede ver cómo es que este funciona en el siguiente diagrama de la parte 1



Ok ahora que ya hemos ejemplificado como se mira el programa es necesario ver que acciones tiene disponible el programa en total son 10 vistas de la siguiente forma:

1. Crear un AFD manualmente
2. Cargar un AFD desde archivo .txt
3. Mostrar la definición formal del AFD
4. Mostrar la tabla de transición
5. Validar la estructura del autómata
6. Evaluar una cadena
7. Evaluar un archivo de cadenas
8. Consultar el historial de evaluaciones
9. Cargar o crear otro autómata
10. Salir

Comencemos aclarando que las opciones se seleccionan desde el menú introduciendo cual es la opción que quiere del menú ahora continuando con la opción 1 que es la forma de crear un AFD manualmente en este caso el programa pedirá las siguientes entradas

```
--- Creación manual de un AFD ---  
Nombre del autómata: automata1  
Estados Q (separados por coma, ej. q0,q1,q2): q0,q1  
Alfabeto Sigma (separado por coma, ej. a,b): a  
Estado inicial q0: q0  
Estados finales F (separados por coma, puede dejarse vacío): q1
```

```
Ingresa las transiciones con el formato origen,simbolo,destino.  
Escribe FIN cuando termines.
```

Ahora para su uso seria de la siguiente forma

1. Solicita el nombre del autómata (no vacío; reintenta hasta que sea válido).
2. Solicita el conjunto de estados Q (lista separada por comas); rechaza conjunto vacío o con duplicados.
3. Solicita el alfabeto Σ (lista separada por comas); rechaza vacío, duplicados o que contenga el símbolo épsilon.
4. Solicita el estado inicial q_0 ; reintenta hasta que pertenezca a Q.
5. Solicita el conjunto de estados finales F (puede quedar vacío); rechaza duplicados o que no sea subconjunto de Q.
6. Solicita transiciones una por una ('origen,simbolo,destino') hasta que el usuario escribe 'FIN'; rechaza en el acto estados/símbolos fuera de Q/Σ , el símbolo épsilon, y transiciones repetidas para el mismo par (origen, símbolo).

7. Construye el objeto `AFD` con `validado=False` y lo devuelve; queda como el autómata actual de la sesión.

Sus datos de entrada son de la siguiente forma:

- Nombre del autómata: `str` / No vacío
- Estados Q : `str` (lista por comas) Se convierte a `set`; sin vacíos ni duplicados
- Alfabeto Σ : `str` (lista por comas) Se convierte a `set`; sin duplicados; prohíbe `ε`/`epsilon`
- Estado inicial q_0 : `str` Debe pertenecer a Q
- Estados finales F : `str` (lista por comas, puede ser `""`) Se convierte a `set`; debe ser subconjunto de Q |
- Transiciones: `str` `"origen, símbolo, destino"`, repetido hasta `"FIN"`
origen/destino $\in Q$, símbolo $\in \Sigma$, sin repetir (origen, símbolo)

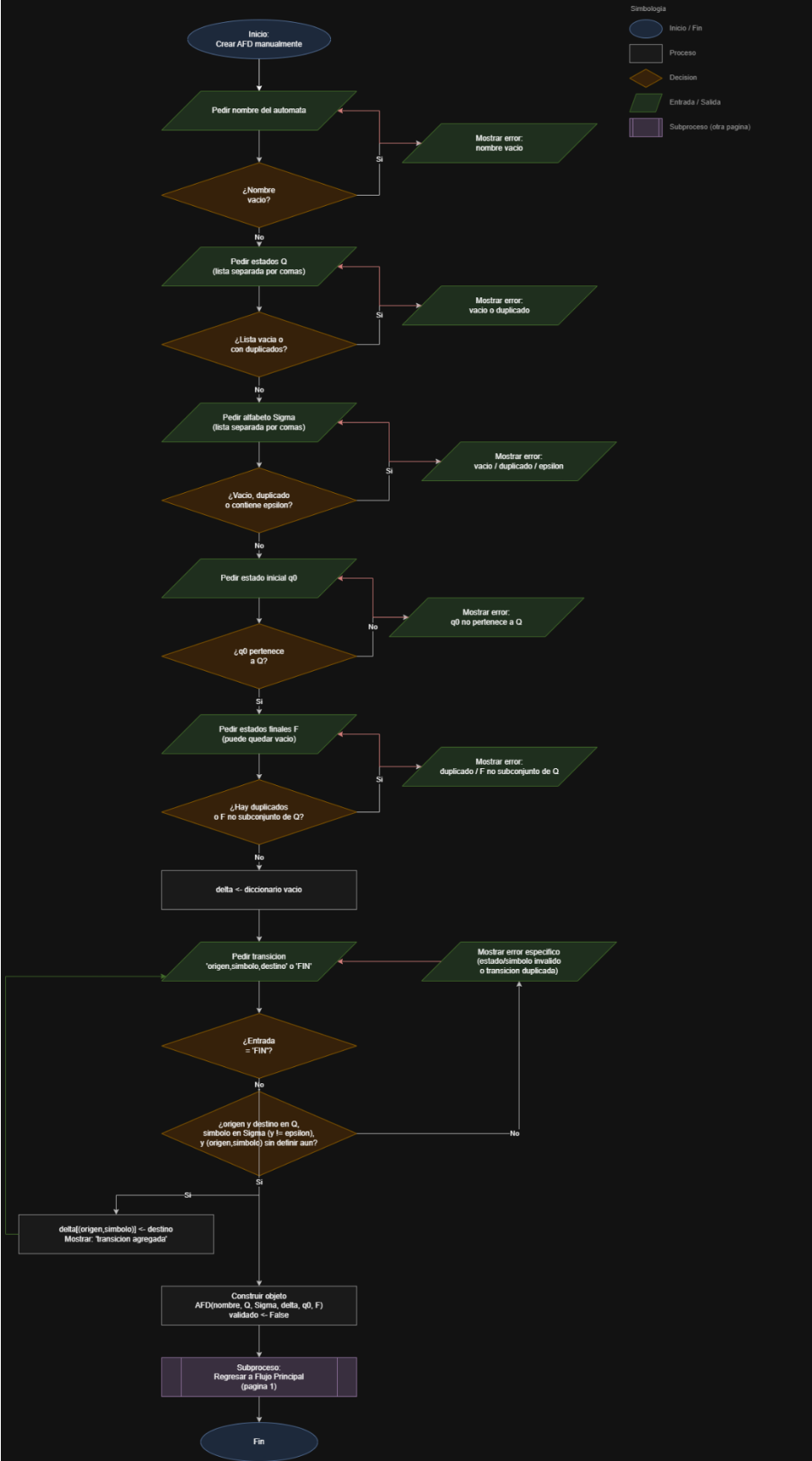
Sus variables son de la siguiente forma

- `nombre: str`, `Q: set[str]`, `sigma: set[str]`, `q0: str`, `F: set[str]`
- `delta: dict[(str, str), str]` — se construye en `\_pedir\_transiciones`
- `elementos: list[str]` — temporal, resultado de `\_separar\_lista()`
- `entrada, partes, origen, símbolo, destino: str` — temporales por cada transición ingresada
- `afd: AFD` — objeto final devuelto

Sus condiciones y cálculos

- Sin duplicados: `len(elementos) == len(set(elementos))`, aplicado a Q , Σ y F .
- Épsilon prohibida: se rechaza si algún elemento de Σ (o el símbolo de una transición) está en `{ε, 'epsilon', 'Epsilon', 'EPSILON', 'ÉPSILON'}`.
- `q0 ∈ Q` (se vuelve a pedir hasta cumplirse).
- `F ⊆ Q` → `F.issubset(Q)`.
- Por transición: `origen ∈ Q`, `destino ∈ Q`, `símbolo ∈ Σ`.
- Unicidad de δ : `(origen, símbolo) not in delta` antes de agregar — evita que el autómata se vuelva un AFND.
- δ se construye como diccionario (`delta[(origen, símbolo)] = destino`), nunca como una cadena de `if/elif` por estado.

Siguiendo con el programa se puede ver expresado con el siguiente grafico



Para la opción 2 se utilizan las mismas funciones que en la opción 1 con una diferencia única que en este caso se hace un llamado a la función de carga de archivo y normalizar las rutas para poder hacerlo en este caso estas son las acciones

1. Solicita la ruta del archivo ``txt``.
2. Normaliza la ruta (``normalizar_ruta_archivo``): quita comillas envolventes, expande variables de entorno y ``~``, unifica separadores.
3. Intenta abrir y leer todas las líneas; si falla (``FileNotFoundError`/`OSError``), reporta el error y regresa sin construir el AFD.
4. Recorre línea por línea, reconociendo cada una con expresiones regulares (``NOMBRE=`, `ESTADOS=`, `ALFABETO=`, `INICIAL=`, `FINALES=`, `TRANSICIONES:`,` y luego cada línea de transición ``origen,simbolo,destino``).
5. Acumula un error de sintaxis (con número de línea) por cada línea que no coincide con ningún patrón, ****sin detener la lectura****.
6. Detecta y rechaza transiciones con símbolo épsilon y transiciones múltiples para el mismo par (origen, símbolo) — indicio de AFND.
7. Verifica que estén presentes los componentes obligatorios (``NOMBRE`, `ESTADOS`, `ALFABETO`, `INICIAL``); si falta alguno, no construye el AFD.
8. Si los componentes mínimos están presentes, construye el objeto ``AFD`` (``validado=False``) y lo devuelve junto con la lista de errores encontrados (puede no estar vacía, aunque el AFD sí se haya construido).

Sus datos de entrada son de la siguiente forma:

- Ruta del archivo ``str`` Debe apuntar a un archivo ``txt`` existente y legible, con el formato ``NOMBRE=`, `ESTADOS=`, `ALFABETO=`, `INICIAL=`, `FINALES=`, `TRANSICIONES:`,`

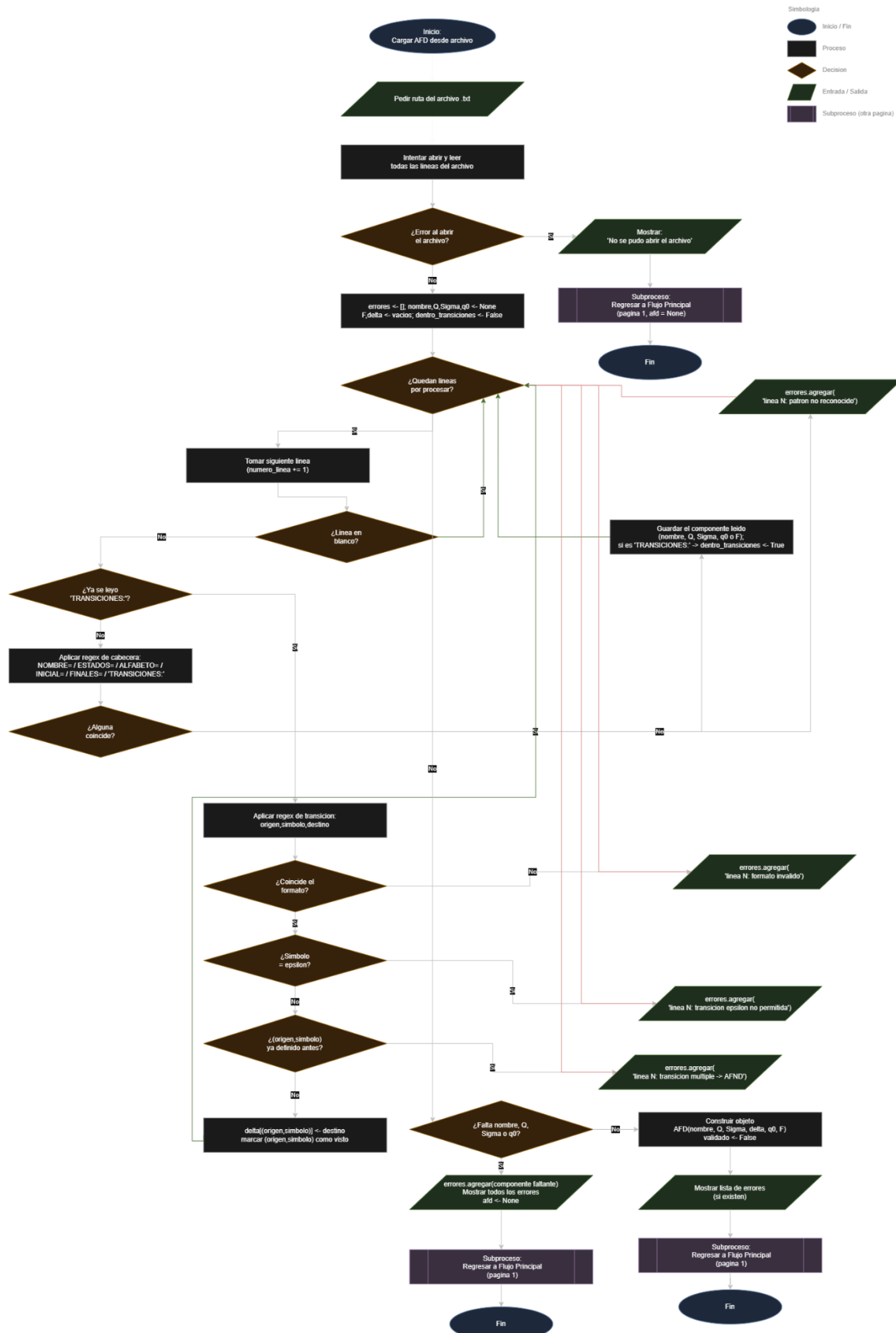
Sus variables son de la siguiente forma

- ``ruta: str`, `líneas: list[str]`, `errores: list[str]`, `numero_linea: int``
- ``nombre: str | None`, `Q: set[str] | None`, `sigma: set[str] | None`, `q0: str | None`, `F: set[str]`, `delta: dict[(str,str), str]``
- ``dentro_de_transiciones: bool`` — indica si ya se pasó la línea ``TRANSICIONES:``
- ``transiciones_vistas: set[(str,str)]`` — detecta pares repetidos
- ``PATRON_NOMBRE, PATRON_ESTADOS, PATRON_ALFABETO, PATRON_INICIAL, PATRON_FINALES, PATRON_ENCABEZADO_TRANSICIONES, PATRON_TRANSICION, PATRON_IDENTIFICADOR: re.Pattern`` — constantes del módulo

Sus condiciones y cálculos

- Cada línea se contrasta, en orden, contra los patrones regex hasta encontrar coincidencia; si ninguno coincide → error `"línea N: no coincide con ningún patrón reconocido"`.
- ``ESTADOS`/`ALFABETO`/`FINALES``: se separan por comas, se valida duplicados (``len(lista)==len(set(lista))``) y nombres válidos (``^[A-Za-z0-9_]+$``).
- Épsilon prohibido: se filtra de Σ y se rechaza como símbolo de transición.
- Transición: ``(origen,simbolo) not in transiciones_vistas`` antes de agregar a δ ; si ya existe, se reporta `"transición múltiple -> AFND"` y se descarta esa línea.
- Construcción final: solo si ``nombre``, ``Q``, ``sigma`` y ``q0`` son distintos de ``None`` (F puede quedar vacío por defecto).

El diagrama de la parte de abajo representa la forma en la que el caso 2 funciona



Lo que hace la opción 3 es mostrar la definición formal del AFD en este caso no habrá diagrama porque no es necesario para llamar las formas que se usan en los casos 1 y 2

Ahora para su uso seria de la siguiente forma:

1. Verifica que haya un autómata cargado (``hay_automata_cargado``); si no, muestra error y regresa al menú.
2. Imprime la quintupla ``M = (Q, Σ, δ, q0, F)`` del autómata actual: Q, Σ y F ordenados alfabéticamente; q0; y cada transición ``δ(estado, símbolo) = destino`` ordenada.

Sus datos de entrada son de la siguiente forma:

Ninguno — usa el autómata ya cargado en memoria.

Sus variables son de la siguiente forma

- ``afd: AFD`` (el autómata actual)
- ``estado, simbolo, destino: str`` — usados al iterar ``sorted(afd.delta.items())``

Sus condiciones y cálculos

- Requiere ``automata_actual != None``.
- Q, Σ, F se muestran con ``sorted()`` (un ``set`` de Python no garantiza orden).
- δ se recorre como ``sorted(afd.delta.items())``, es decir, ordenado por la clave (estado, símbolo).

Para la opción 4 de mostrar la tabla de transición no habrá entrada debido a que solo muestra datos precargados ni tampoco diagrama porque no es necesario ya que se muestra en las opciones 1 y 2 al cargar los autómatas o al crearlos

Ahora para su uso seria de la siguiente forma:

1. Verifica que haya un autómata cargado.
2. Calcula el ancho de columna necesario según el nombre de estado más largo.
3. Imprime un encabezado con cada símbolo del alfabeto como columna.
4. Imprime una fila por cada estado (marcando ``->`` si es el inicial y ``*`` si es final), con el destino ``δ(estado, símbolo)`` en cada columna, o ``-`` si no está definida.

Sus datos de entrada son de la siguiente forma:

Ninguno

Sus variables son de la siguiente forma

- ``estados: list[str]`` — ``sorted(afd.Q)``
- ``simbolos: list[str]`` — ``sorted(afd.sigma)``
- ``ancho_estado: int``, ``encabezado: str``, ``fila: str``, ``etiqueta: str``, ``marcas: str``
- ``destino: str`` — ``afd.delta.get((estado, simbolo), "-")``

Sus condiciones y cálculos

- Requiere ``automata_actual != None``.
- ``ancho_estado = max(len(e) for e in estados + ["Estado"]) + 3``.
- Por cada estado: ``marcas = "->"`` si ``estado == q0``, más ``"*"*`` si ``estado ∈ F``.
- ``destino = afd.delta.get((estado, simbolo), "-")`` — mismo mecanismo genérico de δ que usa la simulación, sin condicionales por estado.

Para el caso 5 la validación de la estructura del autómata es necesario hacer un gráfico para ejemplificar cómo funcionan las diferentes funciones que llama debido a que ejecuta el motor de validación en los autómatas cargados y especifica si hay errores de estructura o si está incompleto.

Ahora para su uso seria de la siguiente forma:

1. Verifica que haya un autómata cargado.
2. Ejecuta el motor de validación (``validar_estructura``): revisa ``q0 ∈ Q``, ``F ⊆ Q``, consistencia de cada transición, y calcula los pares (estado, símbolo) sin transición definida (``incompletas``).
3. Si hay errores estructurales graves, los muestra todos, marca ``validado=False`` y no continúa.
4. Si el AFD está incompleto (sin errores graves pero con pares faltantes), muestra la lista de transiciones faltantes y pregunta si se desea completar con un estado de trampa.
 - a. Si el usuario responde ``S``: crea el estado de trampa (con nombre único, ej. ``q_trampa``, ``q_trampa_2``...), agrega ``δ[(estado,símbolo)] = q_trampa`` para cada par faltante y ``δ[(q_trampa,símbolo)] = q_trampa`` para cada símbolo (auto-transición); vuelve a validar.
 - b. Si responde cualquier otra cosa: deja ``validado=False`` y no continúa.
5. Si no hay errores ni pares faltantes (desde el inicio o tras completar la trampa): marca ``validado=True`` y ejecuta el análisis estructural (``analizar_estructura``): estados alcanzables desde `q0` (recorrido sobre δ), inaccesibles (``Q - alcanzables``), finales alcanzables (``F ∩ alcanzables``) y si el lenguaje podría ser vacío.

Sus datos de entrada son de la siguiente forma:

Confirmación de estado de trampa `str` (`S`/`N`) Solo se solicita si el AFD resultó incompleto

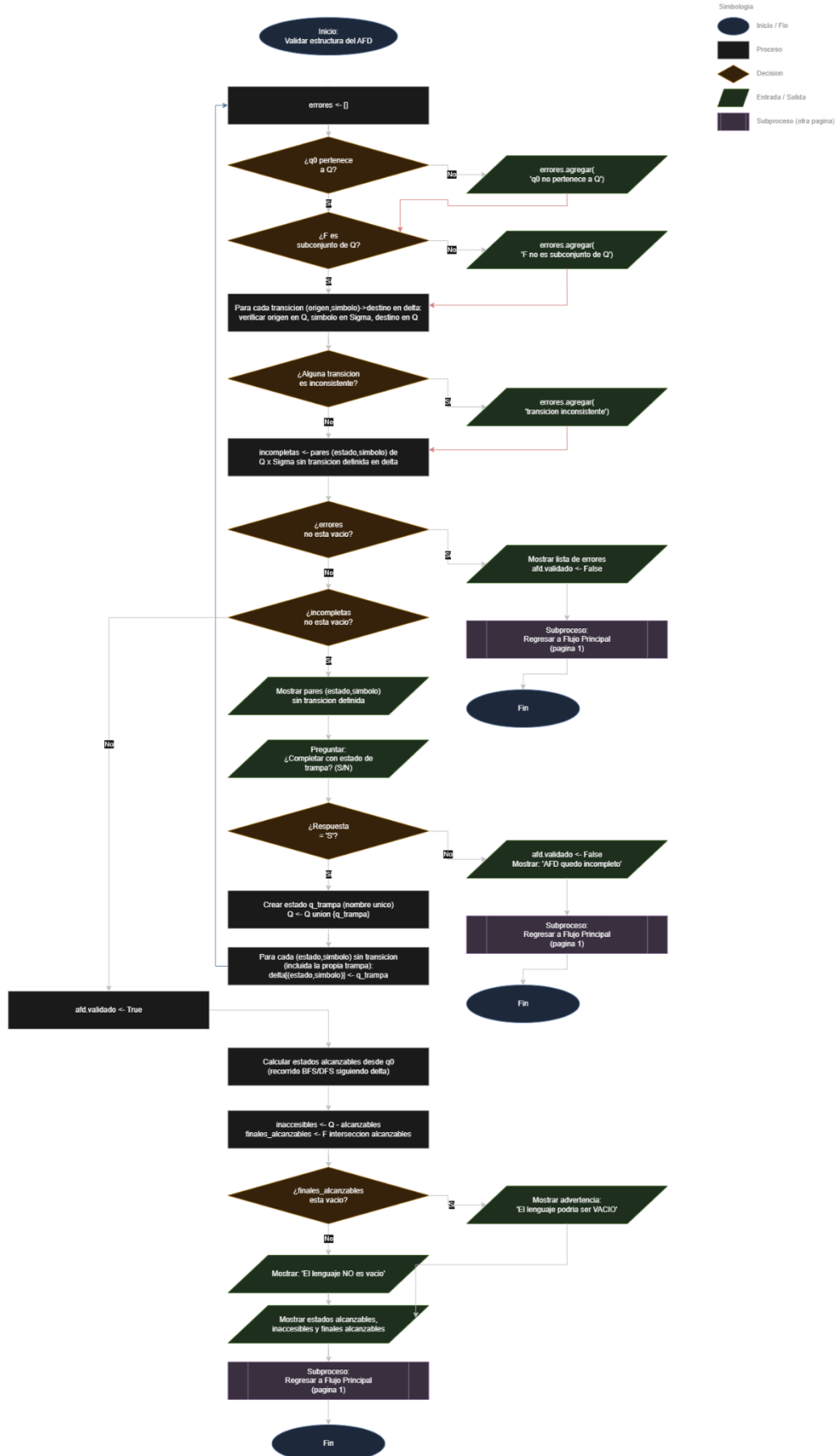
Sus variables son de la siguiente forma

- `errores: list[str]`, `incompletas: list[(str,str)]`
- `respuesta: str`, `nombre\_trampa: str`, `sufijo: int`
- `analisis: AnalisisEstructural` — `alcanzables: set`, `inaccesibles: set`,
`finales\_alcanzables: set`, `lenguaje\_vacio: bool`
- `alcanzables: set[str]`, `pendientes: list[str]` — pila del recorrido

Sus condiciones y cálculos

- $q_0 \in Q$; si no, error.
- $F \subseteq Q$ ($F - Q$ debe ser $\emptyset$); si no, error listando los elementos que faltan.
- Por cada $(origen, simbolo) \rightarrow destino$ en δ : $origen \in Q$, $simbolo \in \Sigma$, $destino \in Q$; cualquier incumplimiento es error.
- $incompletas = [(estado, simbolo) \text{ para } estado \in Q, simbolo \in \Sigma \text{ si } (estado, simbolo) \text{ not in } \delta]$ — verificación de determinismo/completitud (Funcionalidad 2, punto 3).
- Estado de trampa: $nombre_trampa = "q_trampa"$; si ya existe en Q , se prueba $"q_trampa_2"$, $"q_trampa_3"$... hasta que sea único. La trampa nunca se agrega a F .
- $alcanzables$: recorrido desde q_0 siguiendo δ para cada símbolo de Σ , acumulando todo estado visitado.
- $inaccesibles = Q - alcanzables$.
- $finales_alcanzables = F \cap alcanzables$.
- $lenguaje_vacio = (len(finales_alcanzables) == 0)$.

En la siguiente pagina se muestra el diagrama para ejemplificar como funciona el caso
5



En el caso 6 que es la parte de evaluar una cadena hay que utilizar las funciones de simular las cadenas y de imprimirlas por lo que es necesario usar un gráfico para ejemplificar

Ahora para su uso seria de la siguiente forma:

1. Verifica que haya un autómata cargado ****y validado**** (`puede\_evaluar`).
2. Solicita la cadena a evaluar (puede dejarse vacía = λ).
3. Simula: para cada símbolo, en orden, si no pertenece a Σ o si δ no está definida para `(estado\_actual, símbolo)`, detiene el proceso y marca "Rechazada" con el motivo; si no, avanza `estado\_actual` y registra el paso en la traza.
4. Si se procesaron todos los símbolos sin error, determina el veredicto según si `estado\_actual  $\in F$ `.
5. Imprime la traza paso a paso (` $\delta(\text{origen}, \text{símbolo}) = \text{destino}$ `) y el veredicto en color (verde = Aceptada, rojo = Rechazada).
6. Agrega el resultado al historial de la sesión.

Sus datos de entrada son de la siguiente forma:

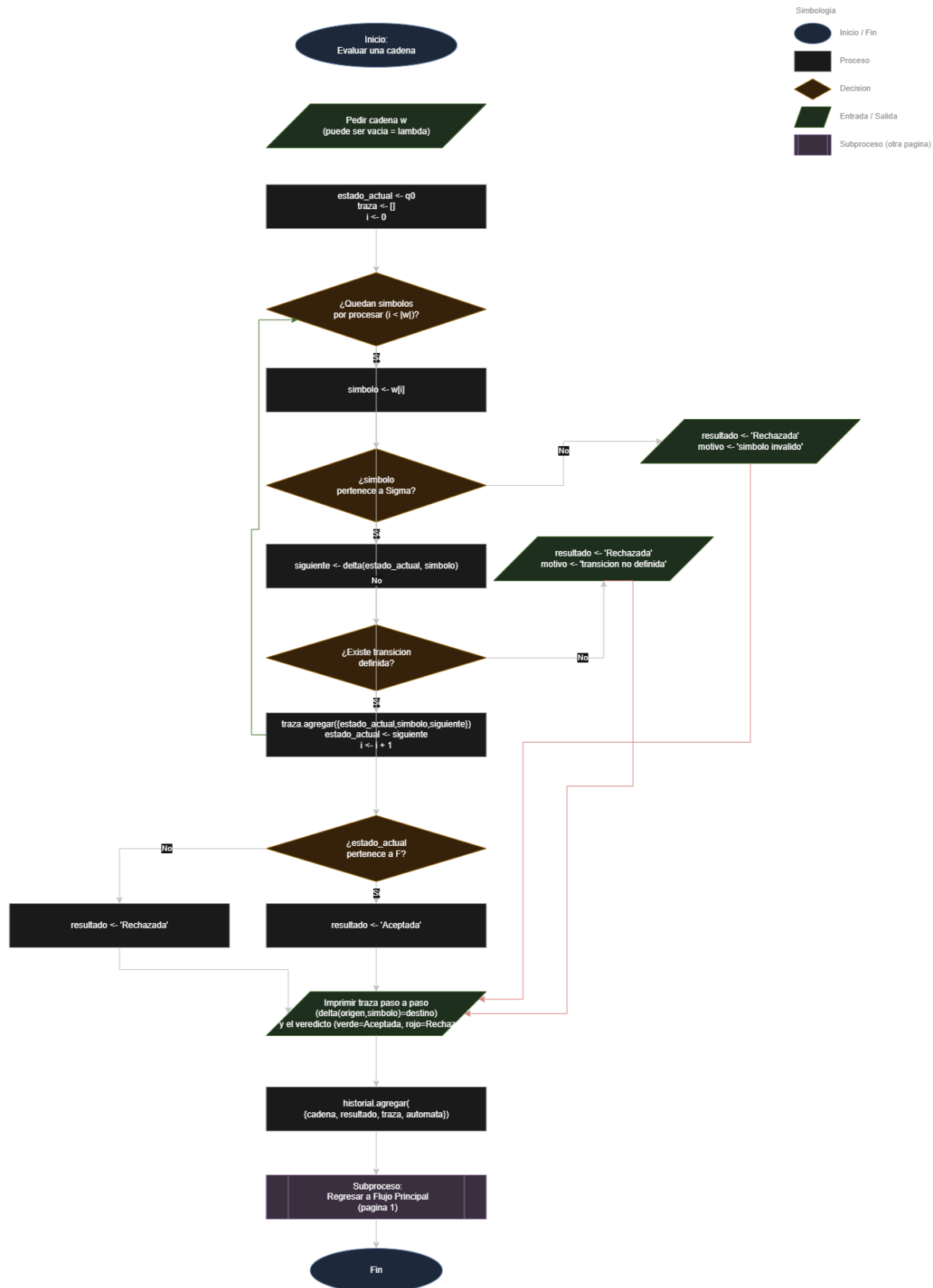
- Cadena para evaluar `str` Puede ser vacía (λ); si contiene un símbolo fuera de Σ , se acepta el ingreso, pero se rechaza con motivo explicado

Sus variables son de la siguiente forma

- `cadena: str`, `estado\_actual: str`
- `traza: list[Paso]` — cada `Paso`: `estado\_origen`, `símbolo`, `estado\_destino`
- `resultado: ResultadoEvaluacion` — `cadena`, `traza`, `veredicto`, `motivo`
- `símbolo, siguiente: str` — temporales del bucle

Sus condiciones y cálculos

- Requiere `automata\_actual != None` y `automata\_actual.validado == True`.
- `estado\_actual  $\leftarrow q_0$ ` al inicio.
- Por cada símbolo `s<sub>i</sub>` de la cadena, en orden: si `s<sub>i</sub>  $\notin \Sigma$   $\rightarrow$  Rechazada (motivo: símbolo inválido); si ` $\delta(\text{estado_actual}, s_i)$ ` no existe  $\rightarrow$  Rechazada (motivo: transición no definida); si no, `estado_actual $\leftarrow \delta(\text{estado_actual}, s_i)$ ` y se agrega el paso a la traza.
- Si no hubo error: `veredicto = "Aceptada"` si `estado\_actual  $\in F$ `, si no `"Rechazada"`.
- Cadena vacía (`w =  $\lambda$ `): no se procesa ningún símbolo; el veredicto depende únicamente de si `q<sub>0</sub>  $\in F$ `.



En el caso 7 es similar al caso 6 debido a que realizaremos lo mismo solo que en este caso será ejecutando muchas cadenas entonces se utiliza el mismo diagrama, pero con el cambio de la entrada y los resultados

Ahora para su uso seria de la siguiente forma:

1. Verifica que haya un autómata cargado y validado.
2. Solicita la ruta del archivo de cadenas (una por línea).
3. Intenta abrir y leer todas las líneas; si falla, reporta el error y regresa al menú sin colapsar.
4. Evalúa cada línea con el mismo algoritmo de la opción 6 (`evaluar\_cadena`), acumulando resultados.
5. Imprime la traza y el veredicto de cada cadena, y las agrega todas al historial.
6. Imprime un resumen: total evaluadas, aceptadas y rechazadas.

Sus datos de entrada son de la siguiente forma:

Ruta del archivo de cadenas `str` Archivo de texto con una cadena por línea (una línea vacía = λ)

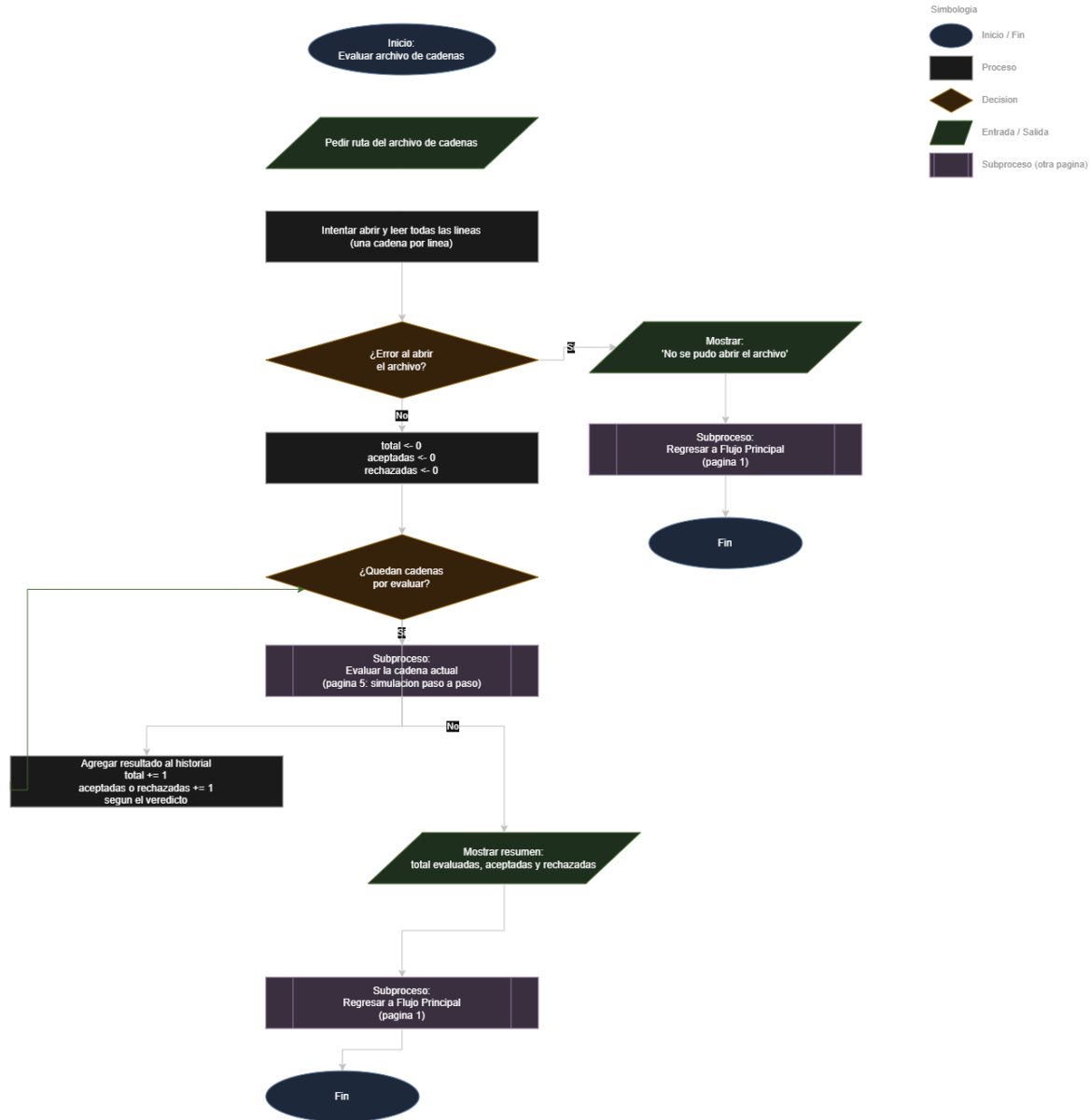
Sus variables son de la siguiente forma

- `ruta: str`, `cadenas: list[str]`
- `resultados: list[ResultadoEvaluacion]`
- `resumen: dict` — `{"total": int, "aceptadas": int, "rechazadas": int}`
- `total\_aceptadas, total\_rechazadas: int` — acumuladores internos de `evaluar\_lote`

Sus condiciones y cálculos

- Requiere `automata\_actual != None` y `validado == True`.
- Lectura protegida con `try/except FileNotFoundError/OSError`.
- Por cada cadena de la lista: se aplica el algoritmo de la opción 6; `total += 1`; `aceptadas += 1` si `veredicto == "Aceptada"`, si no `rechazadas += 1`.

En la siguiente pagina se vera el diagrama de la opción 7



Para la opción 8 no es necesario un diagrama debido a que solo es una consulta de la información de las evaluaciones de los AFD que se han echo

Ahora para su uso seria de la siguiente forma:

1. Si no se ha evaluado ninguna cadena en la sesión, muestra un mensaje informativo.
2. Si hay registros, lista cada uno numerado, mostrando el autómata usado, la cadena (o `"λ (cadena vacía)"`) y el veredicto en color.

Sus datos de entrada son de la siguiente forma:

Ninguno

Sus variables son de la siguiente forma

- ``historial: Historial`` — envuelve ``_registros: list[RegistroHistorial]``
- ``RegistroHistorial`: `cadena: str`, `veredicto: str`, `traza: list[Paso]`, `automata: str``
- ``i, registro`` — variables de iteración (``enumerate``)
- ``cadena_mostrar, resultado_coloreado: str`` — temporales de formato

Sus condiciones y cálculos

- No requiere ``automata_actual`` cargado — el historial es independiente del AFD activo en ese momento.
- Persiste aunque se descarte o cambie el autómata (opción 9): solo se reinicia al cerrar el programa.
- ``cadena_mostrar = "λ (cadena vacía)"` si ``registro.cadena == ""`, si no el texto tal cual.

Para la opción 9 solo es descartar toda la información que tienen las variables de entrada y volver a habilitar las opciones 1 o 2 para desbloquear de la opción 3 a la 7

Ahora para su uso sería de la siguiente forma:

1. Descarta el autómata actual (``afd < None``).
2. Muestra un mensaje confirmando que fue descartado.
3. Regresa al menú, donde las opciones 3-7 vuelven a bloquearse hasta usar 1 o 2 de nuevo.

Sus datos de entrada son de la siguiente forma:

Ninguno

Sus variables son de la siguiente forma

- ``afd`` — se resigna a ``None`` en el bucle principal de ``main()``.

Sus condiciones y cálculos

- ``afd < None``.

- El historial ****no**** se modifica (sigue acumulando entradas de autómatas distintos, cada una etiquetada con el nombre del AFD que se usó).

Por último, la opción 10 salir solo termina el programa entonces no hay Datos no hay variables ni condiciones ni cálculos solamente el apartado de acciones

Ahora para su uso sería de la siguiente forma:

1. Imprime un mensaje de despedida (``¡Hasta luego!``).
2. Rompe el bucle principal (``while True``) y el programa termina de forma controlada.

Ahora ya que tenemos la base o de lo que fue parte 1 a continuación este es el nuevo menú para la parte 2

MOTOR DE CONVERSIÓN AFND -> AFD Y SIMULACIÓN

1. Crear un AFD manualmente
2. Cargar un AFD desde archivo .txt
3. Crear un AFND manualmente
4. Cargar un AFND desde archivo .txt
5. Mostrar la definición formal y la tabla del autómeta cargado
6. Validar la estructura del autómeta
7. Convertir el AFND cargado en un AFD equivalente
8. Mostrar la tabla de equivalencias de macroestados
9. Mostrar la tabla de transición del AFD generado
10. Evaluar una cadena
11. Evaluar un archivo de cadenas
12. Consultar el historial de evaluaciones
13. Realizar el análisis estructural
14. Cargar o crear otro autómeta
15. Salir

Diagrama 1. Crear un AFD manualmente

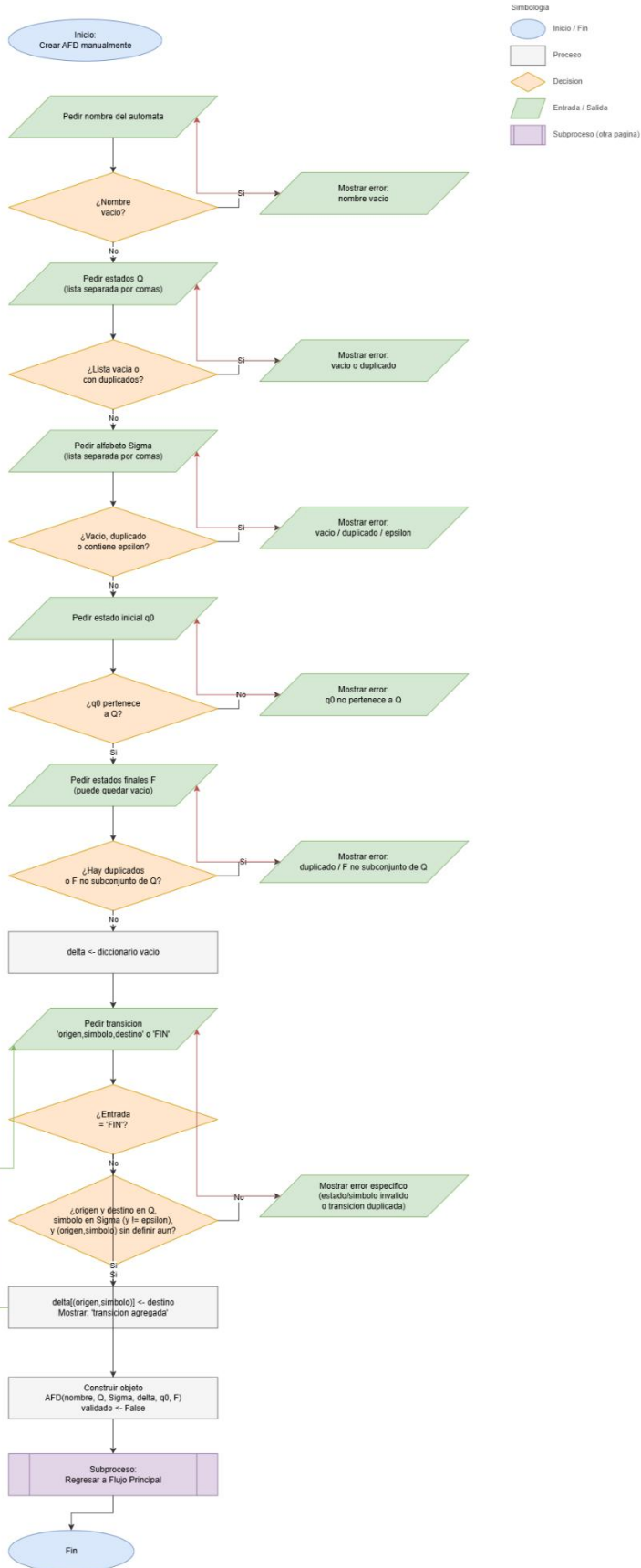


Diagrama: opción 2 Cargar un AFD desde archivo .txt

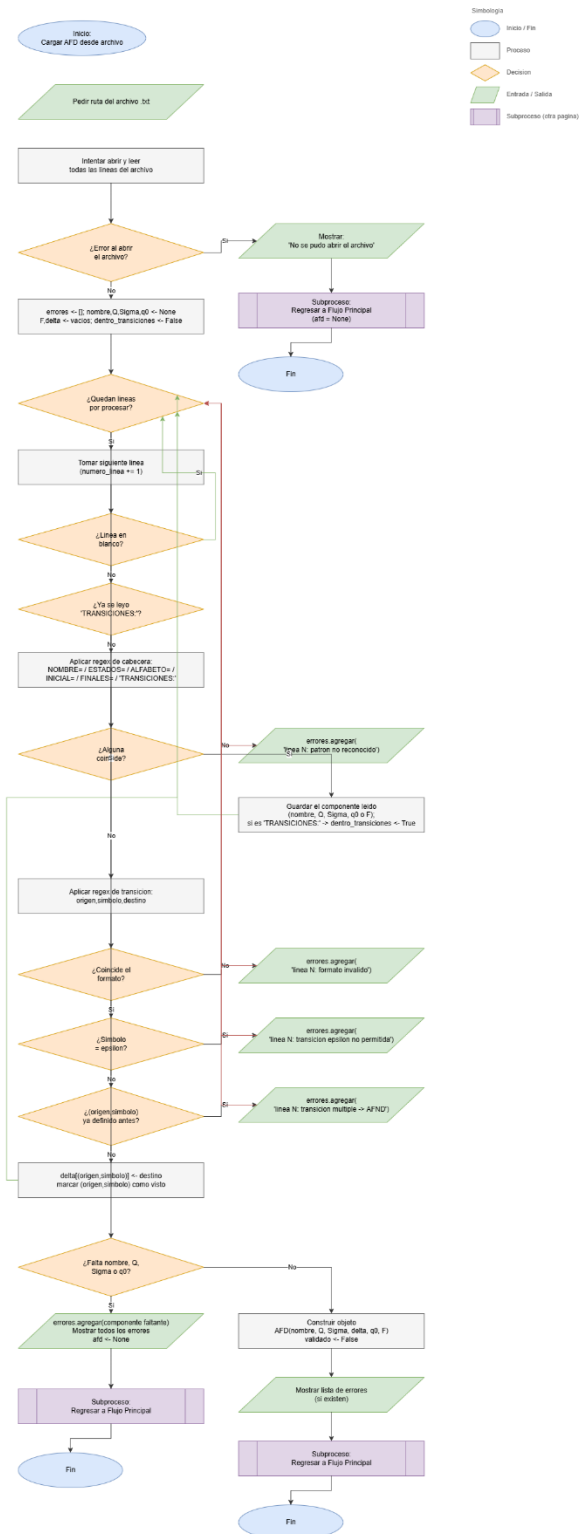


Diagrama opción 3: Crear un AFND manualmente

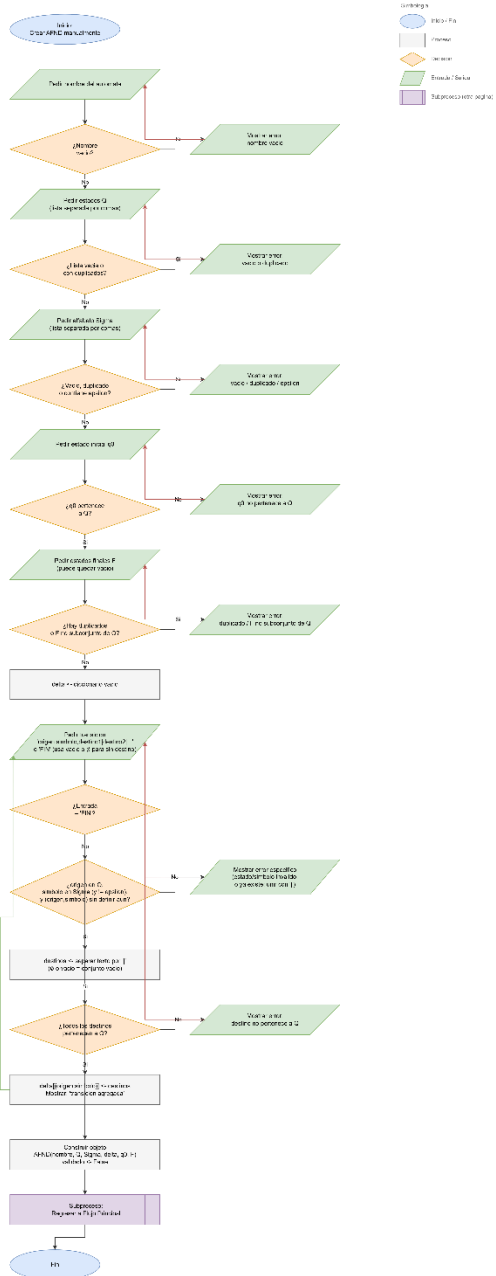


Diagrama opción 5: Mostrar la definición formal y la tabla del autómata cargado

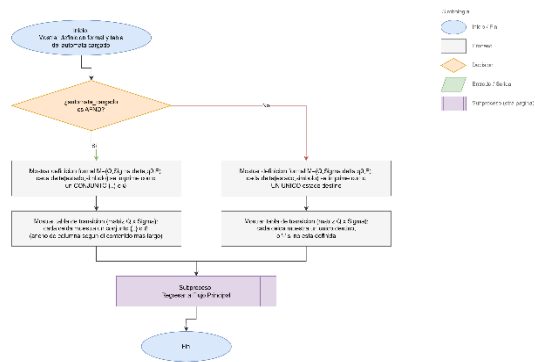


Diagrama opción 6: Validar la estructura del autómata AFD

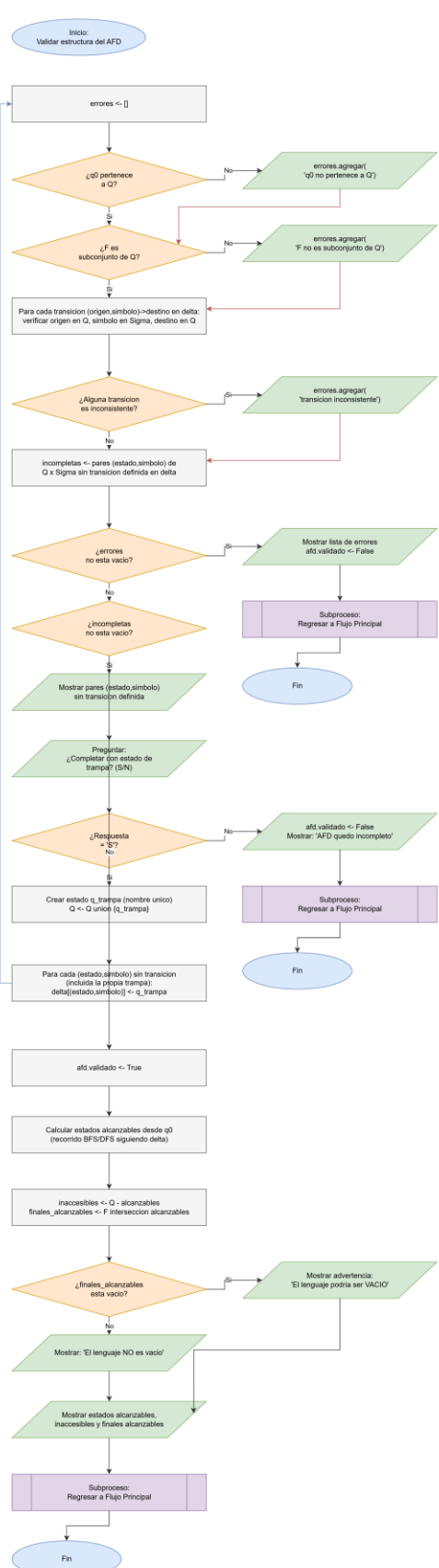


Diagrama opción 6: Validar la estructura del autómata AFND

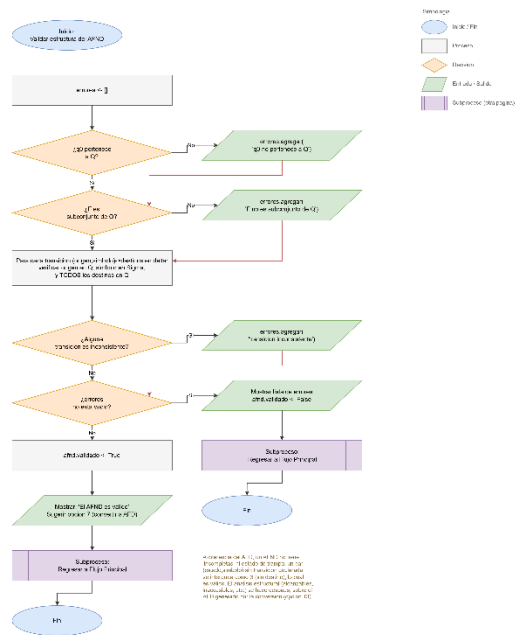


Diagrama opción 7: Convertir el AFND cargado en un AFD equivalente

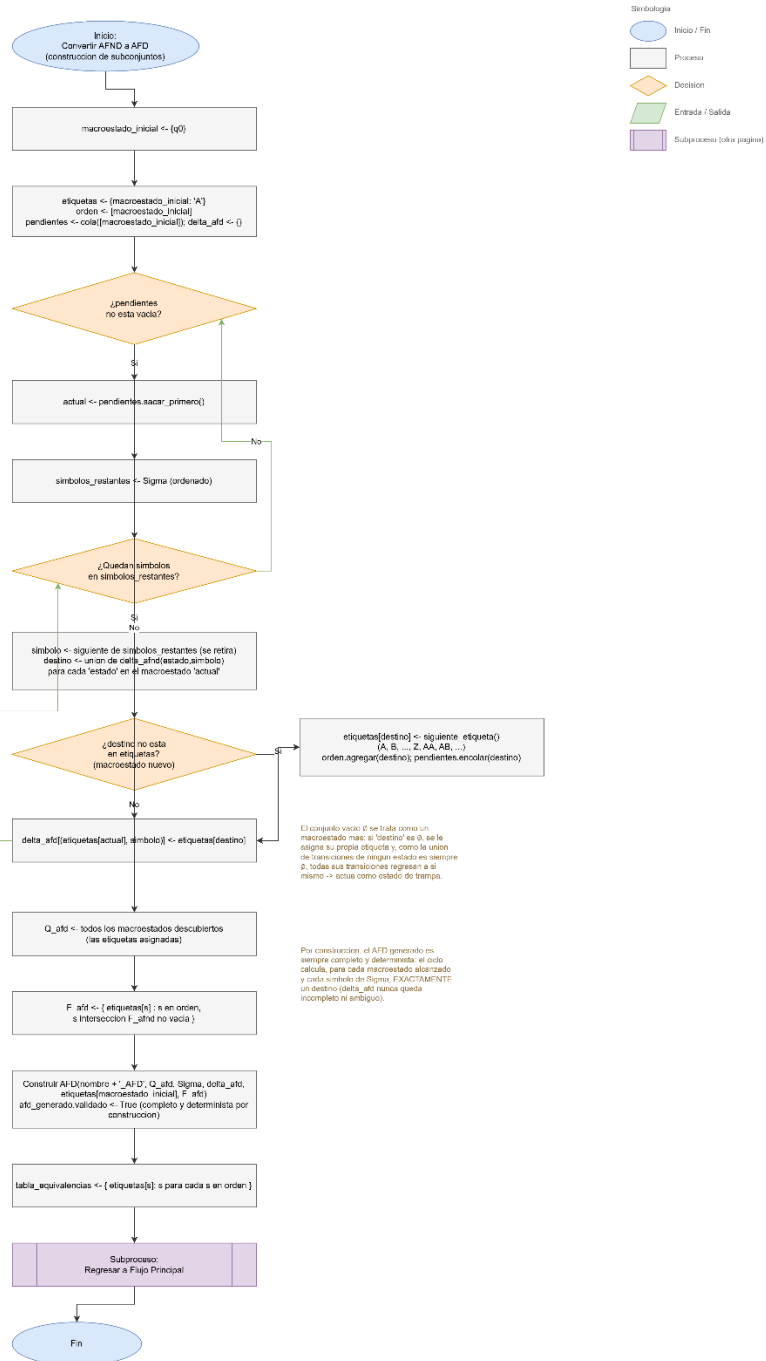


Diagrama opción 8: Mostrar la tabla de equivalencias de macroestados

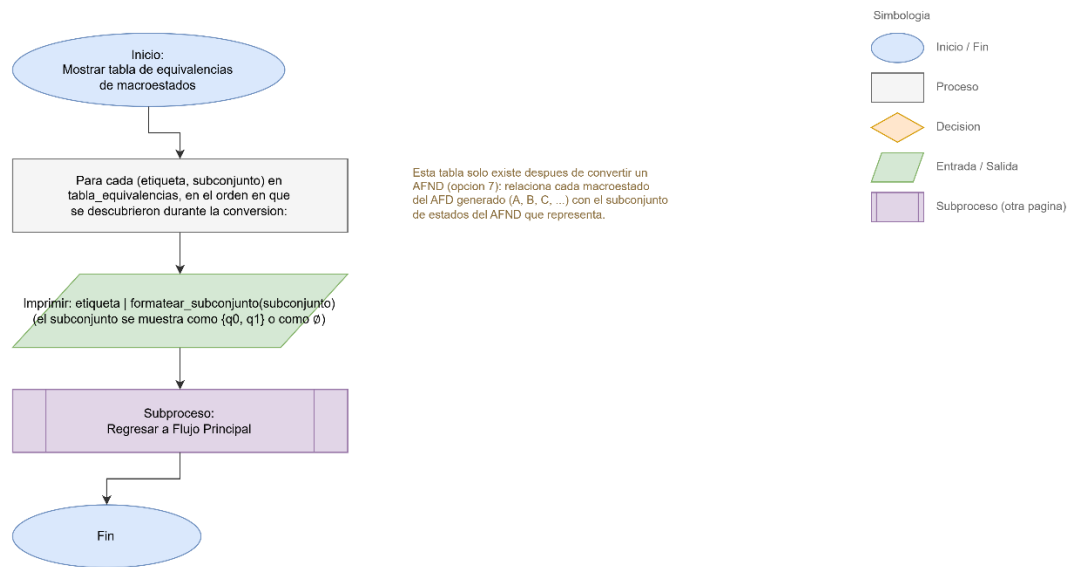


Diagrama opción 9: Mostrar la tabla de transición del AFD generado

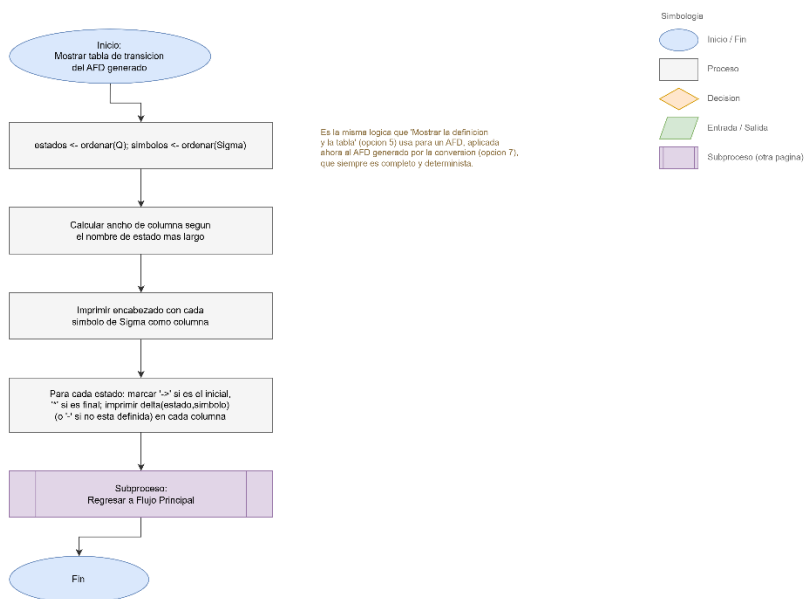


Diagrama opción 10 : Evaluar una cadena

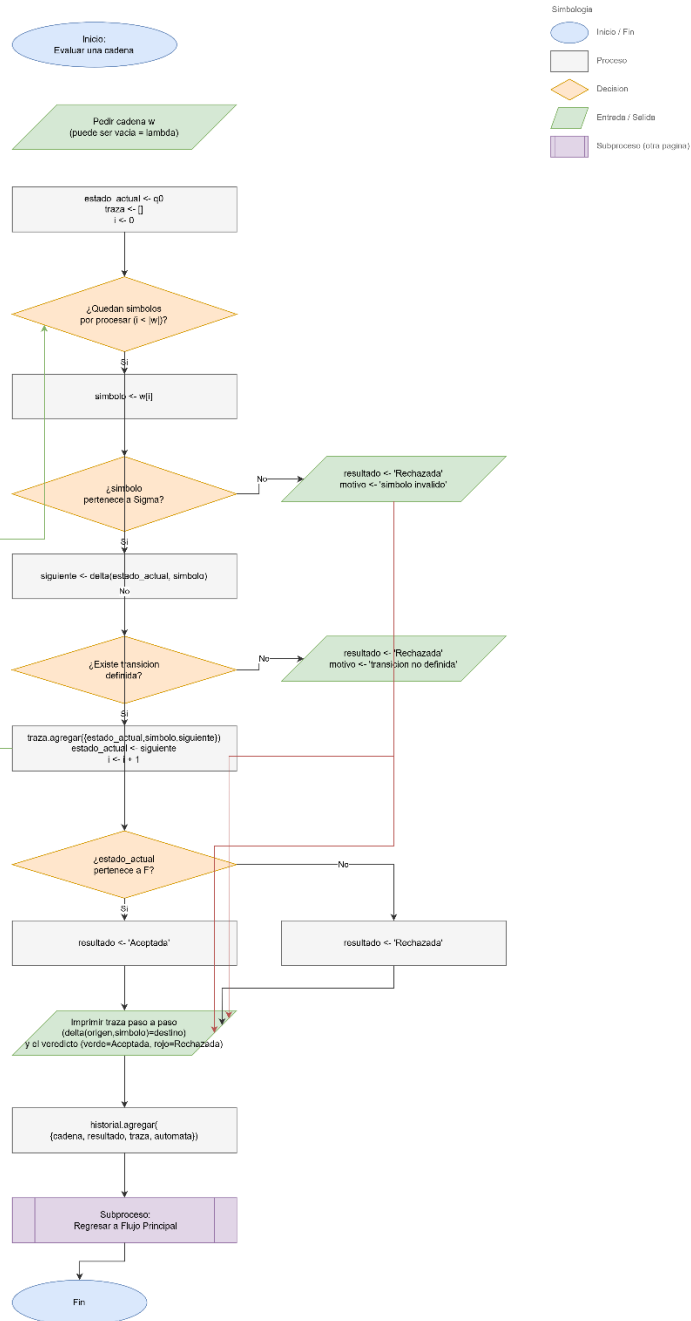


Diagrama opción 11: Evaluar un archivo de cadenas

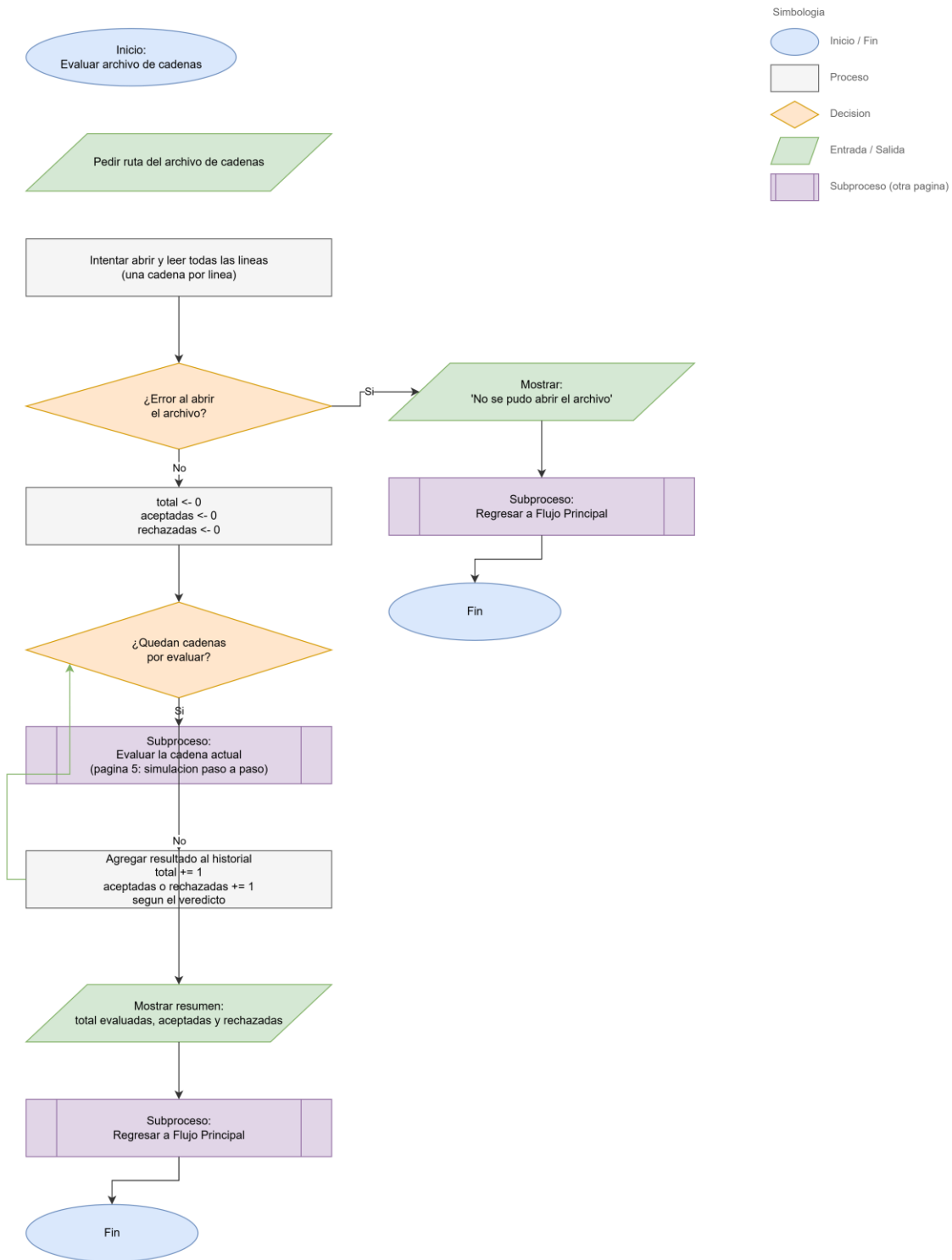


Diagrama opción 12: Consultar el historial de evaluaciones

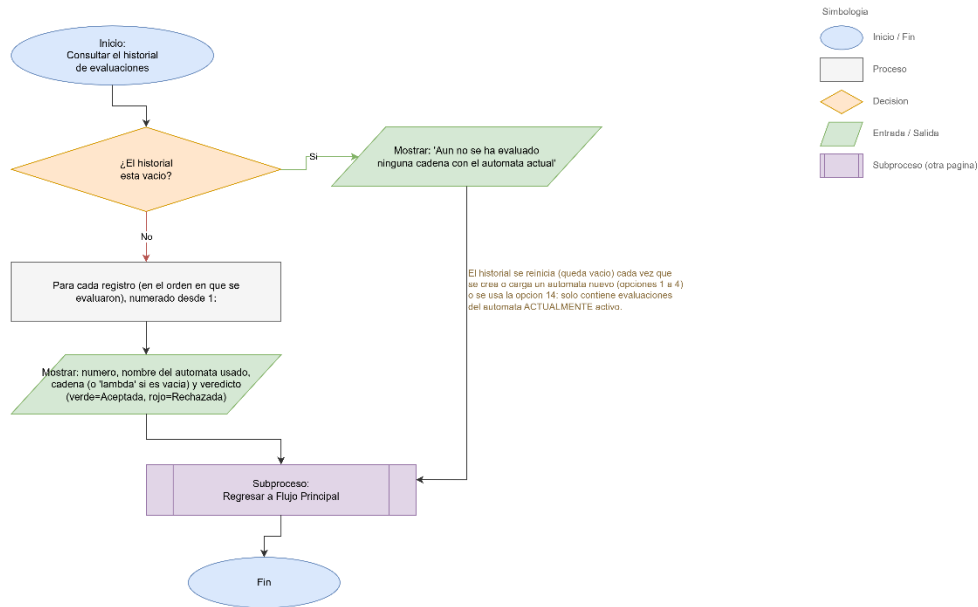


Diagrama opción 13: Realizar el análisis estructural

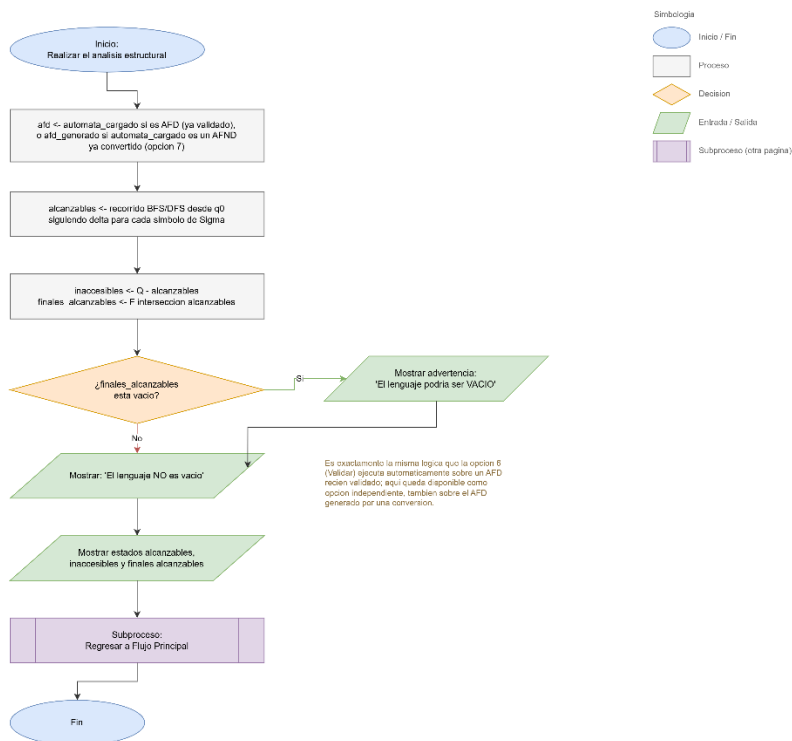


Diagrama opción 14: Cargar o crear otro autómata

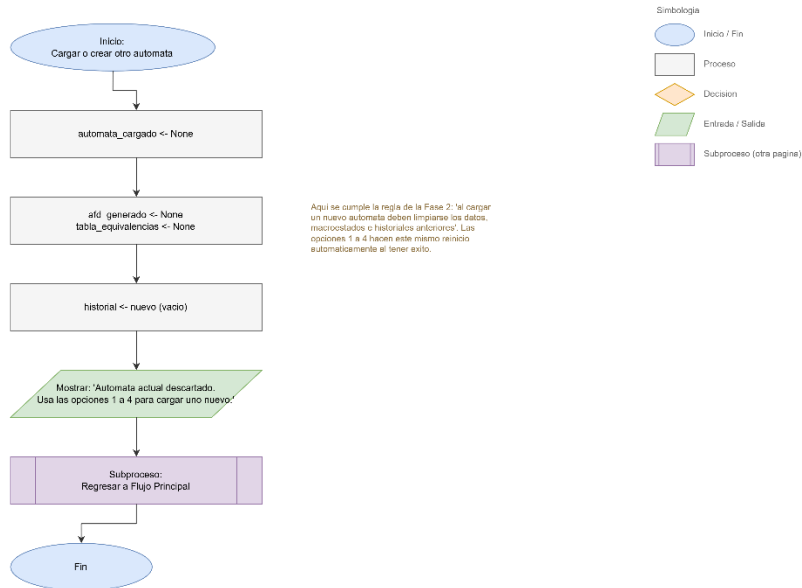


Diagrama opción 15: Salir



Nota: (Todos los Diagramas fueron realizados en draw.io=

